Proof Carrying Execution

A Protocol for Autonomous Action Authorization

Derek A. Hone
Remnant Fieldworks Inc. — Westerville, Ohio
derek@ownerremnantfieldworks.com
executionproof.io

August 2026 — Working Specification v0.2
Status: Pre-publication draft. Independent review and formal standardization are future steps.
Revision note: v0.2 corrects architectural role naming (Sections 5a/5b), reconciles ProofRecord emission responsibility with the EXECUTE_ENABLE standard, and reclassifies `intent_lineage` from required-with-MAY-check to recommended-pending-verification-specification.

Abstract

We propose Proof Carrying Execution (PCE), a protocol requiring that every consequential autonomous action carry independently verifiable proof of its authorization at the moment of execution. A consequential action is a machine-initiated action that produces an effect on an external system, physical state, financial record, or persistent data store that is not automatically reversed by the action itself and would require human or machine intervention to undo. A compliant execution gate refuses any action that does not present a valid PCE package — one that cryptographically binds the proposed action to its authority chain, supporting evidence, policy state, originating intent lineage, constraint envelope, and freshness proof. The gate verifies the package, confirms the post-quantum signature, checks replay resistance, and only then releases execution. Failure at any verification step results in denial; ambiguity is not permission.

PCE extends the intellectual lineage of Proof Carrying Code (Necula, 1997) from the domain of type-safe computation to the domain of consequential autonomous action. Where PCC asked is this code safe to execute?, PCE asks should this proposed change to the world be allowed to happen?

We describe the PCE package format, the verifier contract, the gate contract, the fail-closed invariant, and the relationship between PCE and the ExecutionProof architecture developed by Remnant Fieldworks Inc., which constitutes a partial implementation of the protocol. We also describe what remains to be formally specified, independently implemented, and standardized.

---

1. The Problem PCE Addresses

1.1 The Unasked Question

Modern computing infrastructure is organized around questions that accumulate in layers:

- The compute layer asks: Can this calculation be performed?
- The network layer asks: Can this request reach its destination?
- The identity layer asks: Is this agent who it claims to be?

- The security layer asks: Is this system protected from intrusion?
- The application layer asks: Can this function be executed?

The deployment of autonomous agents into consequential domains — finance, healthcare, infrastructure, industrial operations, physical systems — has exposed a question that no existing layer answers:

Should this proposed change to the world be allowed to happen?

This is not a question about identity. An authenticated agent may still propose an unauthorized action. It is not a question about capability. A capable system may still act outside its sanctioned envelope. It is not a question about security in the traditional sense. A secure perimeter protects a system from outside intrusion; it does not govern what an autonomous agent inside that perimeter is permitted to cause.

The gap is a missing layer: the consequence layer.

1.2 What Constitutes a Consequential Action

A consequential action, for the purposes of this specification, is a machine-initiated action that satisfies both of the following conditions:

First, it produces an effect on an external system, physical state, financial record, or persistent data store. Pure computation, internal state management, and read-only queries do not qualify unless they are the direct precondition of a subsequent consequential action.

Second, the effect is not automatically reversed by the action itself. An action whose consequences persist after the action completes — and that would require separate human or machine intervention to undo — is consequential. Ephemeral side effects that self-reverse are not.

This boundary will require refinement as the protocol matures and as domain-specific profiles are defined. A consequence classifier (the Universal Consequence Classifier concept) is a logical companion to this protocol for systems that must determine at runtime whether a proposed action crosses the consequential threshold.

1.3 The Trust Problem in Autonomous Systems

When a human performs a consequential action, accountability is structured through law, contract, identity, and evidence gathered after the fact. The human is known; the action is traceable; liability exists.

When an autonomous agent performs a consequential action, the current infrastructure offers this:

- A log, if the system is instrumented
- An audit trail, if the system is compliant
- An API key, which grants broad rights for extended periods
- A permission check, which answers is the role permitted this action type but not is this exact action authorized right now by this evidence

None of these constitute pre-execution proof. The agent acts; the record is reconstructed. The question of whether the action should have been released is asked retroactively, if it is asked at all.

PCE proposes to change the ordering. The proof comes before execution. The gate verifies before it releases. The record is generated at the moment of decision, not reconstructed afterward.

---

2. The Intellectual Lineage

2.1 Proof Carrying Code

George Necula introduced Proof Carrying Code (PCC) in the landmark 1997 POPL paper, building on earlier work with Peter Lee on safe kernel extensions.

The problem Necula addressed was the trust problem in mobile code. When a system receives code from an untrusted source — a compiler, a remote machine, a vendor — it faces a dilemma. It can trust the source, which is unsafe. It can re-verify the code itself, which is expensive. Or it can require the code to carry its own safety proof, which the receiving system can check mechanically, cheaply, and without trusting the source.

PCC chose the third path. The code carries the proof. The verifier checks the proof. If the proof is valid, execution is permitted. The trust question moves from do we trust the compiler? to is the proof valid? — and the latter can be answered without trusting anyone.

PCC became foundational. It influenced type-safe language design, sandboxing architectures, the JVM bytecode verifier, and a generation of work in certified compilation and software safety.

2.2 The PCE Extension

PCE applies the same architectural pattern to autonomous action: proof travels with the artifact; the verifier checks proof rather than trusting the source.

The analogy is structurally strong. PCE is not a mechanical extension of PCC’s type-theoretic machinery. It applies the same pattern to a different domain with different security properties and a different trust model — notably, the PCE verifier is stateful (it maintains a nonce registry, heartbeat state, and policy version), whereas PCC verifiers are typically stateless. Key differences are noted where they affect design decisions.

	Proof Carrying Code	Proof Carrying Execution
Domain	Computation	Consequential action
The untrusted source	Compiler / remote system	Autonomous agent
What is carried	Type-safety proof	Authorization proof
What the verifier checks	Safety invariants	Authority, evidence, policy, constraints, freshness
What is refused	Unsafe code	Unauthorized action
Trust moves from	“Do we trust the compiler?”	“Do we trust the agent?”
Trust moves to	“Is the proof valid?”	“Is the proof valid?”
Governing question	Is this code safe to run?	Should this change to the world happen?
Verifier state	Typically stateless	Stateful (nonce registry, policy, heartbeat)
Proof generator	Code producer (external)	Verifier (after evaluating proposer’s inputs)

The lineage is not incidental. It suggests PCE may follow a similar trajectory: from proposed protocol to systems primitive to infrastructure standard. PCC took approximately fifteen years to move from the 1997 paper to widespread embedded influence. The timeline for PCE will depend on adoption pressure, which autonomous agent deployment is generating now.

---

3. The Three-Party Model

PCE defines three distinct architectural roles. No role may substitute for another.

The Proposer. The autonomous agent or system that wishes to execute a consequential action. The proposer submits the proposed action and supporting inputs to the verifier. The proposer does not generate the PCE package; it supplies the raw material from which the verifier constructs it.

The Verifier. The independent component that evaluates the proposed action against authority, evidence, policy, constraints, and freshness requirements. The verifier generates the PCE package, signs it, and emits the ProofRecord. The verifier is the trust anchor of the protocol. In an ExecutionProof deployment, the verifier is the ExecutionProof engine stack. The verifier must satisfy the requirements in §5a.

The Gate. The component that receives the PCE package, verifies the signature, checks freshness and expiration, confirms the verifier_decision, and releases or withholds execution accordingly. The gate does not re-evaluate the action; it verifies the proof. The gate must satisfy the requirements in §5b.

This separation is architecturally critical. The verifier decides; the gate enforces. A system in which the same component both decides and enforces without independent oversight is not PCE-compliant.

---

4. The PCE Invariant

Everything that follows in this specification is an elaboration of a single sentence:

> **An action may not produce consequence unless it presents, at the moment of execution, independently verifiable proof that it is authorized, evidence-supported, within its stated constraints, and fresh.**

This is the PCE invariant. Every design decision in the protocol either supports this invariant or is excluded.

Four conditions are required. Authorization establishes that the appropriate authority has sanctioned this action. Evidence establishes that the factual basis for the action is real and current. Constraints establish that the action falls within its sanctioned limits. Freshness establishes that the proof is not stale and has not been replayed.

All four must be present. The absence of any one is sufficient grounds for denial.

---

5. The PCE Package Format

A PCE package is the unit of proof that an action carries. Every consequential action presented to a compliant gate must arrive with a complete, valid PCE package. A gate that accepts any action without a valid package is not PCE-compliant.

5.1 Package Generation

Package generation is the verifier's responsibility. The proposer submits the proposed `action`, its `actor_id`, its claimed `evidence_set`, and its `constraint_envelope` to the verifier. The verifier evaluates these inputs, populates all remaining fields, signs the complete package with ML-DSA-65, and returns it to the proposer. The proposer presents the signed package to the gate. The proposer cannot generate its own valid package; the verifier signature would be absent or invalid.

5.2 Required Fields

The package contains eleven required fields and one recommended field.

`action` (required)

The exact proposed action, in a form that admits no ambiguity about what consequence would be produced. The binding between proof and action is one of the core security properties of PCE. A proof that could apply to multiple actions is not a valid PCE proof.

`actor_id` (required)

The cryptographically verifiable identity of the agent proposing the action. Not a username or API key. A cryptographic identity that answers: which agent, which model, which version, deployed by whom, under whose authority, governed by which policy at the time of this proposal.

`authority_binding` (required)

The chain of authorization demonstrating that the actor is permitted to propose this exact action. Narrowly scoped, time-bounded, and action-specific — not a role or permission tier. The minimum authorization required for this action and no more.

`evidence_set` (required)

The set of current, verifiable facts that make the action appropriate. Evidence must be current; stale evidence is not valid evidence. Evidence that would support a different action does not satisfy this field.

`policy_state` (required)

The policy under which the action was evaluated, at the time of evaluation. Enables the gate and future auditors to verify that the evaluation was conducted under a policy still in force.

`constraint_envelope` (required)

The authorized boundaries of the action: maximum financial magnitude, maximum scope, specific resources authorized, geographic or temporal limits, blast radius bounds, irreversibility classification, and escalation requirements.

`freshness_nonce` (required)

A one-time cryptographic value generated at package creation and consumed at gate verification. A gate that receives a nonce it has already seen is receiving a replayed package. Replay is denied regardless of all other fields.

`expiration` (required)

The time window after which the package is no longer valid regardless of other fields. A PCE package is not a standing authorization. After this window closes, the package expires and a new package must be generated.

`verifier_decision` (required)

The ALLOW, HOLD, or DENY decision produced by the verifier. The three-way trichotomy is deliberate. HOLD is not a failure; it is a suspension pending resolution — additional approvals, updated evidence, human review. A verifier that returns only ALLOW or DENY cannot express the full range of appropriate responses to a contested action.

`verifier_id` (required)

The cryptographic identity of the verifying authority that evaluated the package and produced the verifier_decision.

`verifier_signature` (required)

A post-quantum cryptographic signature binding all prior fields. Computed over a canonical serialization of all ten prior fields. Any modification to any field after signing invalidates the signature. The gate verifies the signature before doing anything else. The algorithm must be post-quantum secure. ML-DSA-65 (NIST FIPS 204) is the current required algorithm for this field.

`intent_lineage` (recommended — see §5.3)

The chain from originating human intent to this proposed action, as a signed representation of the originating instruction and the transformation steps through which it passed.

5.3 Status of `intent_lineage` in This Version

`intent_lineage` addresses intent drift — the phenomenon in which an action several steps downstream from a human instruction no longer corresponds to what the human intended. This is the PCE implementation of Proof of Human Intent.

In this version of the specification (v0.2), `intent_lineage` is a **recommended** field rather than a required one. Two reasons: first, the verification method for intent drift — determining whether a proposed action is consistent with a declared lineage — is still under active research, and mandating a MUST-check requirement without specifying the verification method creates a compliance loophole worse than the optional field. Second, intent lineage chains through multi-agent systems have no current standardized representation format.

The intent of the authors is for `intent_lineage` to become a required field in a future version of this specification, once the verification method is specified and independently validated. RF's FIDELITY research series (IF-01 through IF-06) is the current internal research lane for this problem. When that specification is ready, it will be incorporated as a required package field with a corresponding verifier obligation.

Until that time, a compliant gate SHOULD log and retain `intent_lineage` when present, but MUST NOT deny an otherwise valid package solely because `intent_lineage` is absent.

---

6. The Verifier Contract

A PCE-compliant verifier must satisfy the following requirements.

Requirement V1 — Evaluate Before Packaging
The verifier MUST evaluate the proposed action against authority, evidence, policy, and constraints before generating a package. Signing a package without evaluation is not compliant.

Requirement V2 — Populate All Required Fields
The verifier MUST populate all eleven required fields before signing. A package with any required field absent MUST NOT be signed.

Requirement V3 — Emit ProofRecord
The verifier MUST emit a tamper-evident ProofRecord at the moment of every evaluation, regardless of outcome — ALLOW, HOLD, or DENY. The ProofRecord is the verifier's permanent artifact. It is independent from the package: it is generated by the verifier and retained by the verifier, whereas the package travels with the action to the gate. A ProofRecord generated after evaluation or after gate release is not compliant.

Requirement V4 — Sign with Post-Quantum Algorithm
The verifier MUST sign the package using ML-DSA-65 (NIST FIPS 204) or a successor algorithm approved for post-quantum assurance.

Requirement V5 — Enforce Expiration
The verifier MUST set an expiration window appropriate to the consequence class of the action. The verifier MUST NOT issue packages with indefinite expiration.

Requirement V6 — Replay Resistance
The verifier MUST generate a unique freshness_nonce for every package. The same nonce MUST NOT appear in two packages.

Requirement V7 — Three-Way Decision
The verifier MUST return one of exactly three decisions: ALLOW, HOLD, or DENY. No other decision state is permitted.

---

7. The Gate Contract

A PCE-compliant gate must satisfy the following requirements. The gate verifies the proof; it does not re-evaluate the action.

Requirement G1 — No Action Without Package
A compliant gate MUST refuse to release execution for any action that does not present a complete PCE package with all eleven required fields present.

Requirement G2 — Signature First
A compliant gate MUST verify the verifier_signature before inspecting any other field. If the signature is invalid, the gate MUST deny the action immediately and log the attempt. It MUST NOT inspect the remaining fields of an unsigned or mis-signed package.

Requirement G3 — Verifier Recognition
The gate MUST verify that the verifier_signature was produced by a verifier it has been provisioned to trust. Recognition is established through a pre-configured cryptographic binding: during provisioning,

the gate stores the public key of its authorized verifier. Each package signature is verified against that key. An assertion that does not carry a valid signature from a provisioned verifier MUST be treated as unsigned and denied under G2. Verifier key rotation and provisioning procedures are defined in §[future Provisioning Annex].

Requirement G4 — Freshness Check
A compliant gate MUST verify that the freshness_nonce has not been previously consumed. If the nonce is already in the gate's seen-nonce set, the gate MUST deny the action and record a replay-attempt event. The seen-nonce set MUST be persistent across gate restarts for the full duration of the expiration window of any consumed nonce.

Requirement G5 — Expiration Check
A compliant gate MUST verify that the current time is within the package's expiration window. An expired package MUST be denied regardless of all other fields.

Requirement G6 — Decision Binding
A compliant gate MUST respect the verifier_decision field.

- DENY: the gate MUST block the action.
- HOLD: the gate MUST suspend the action pending external resolution. It MUST NOT release a HOLDed action without a subsequent re-evaluation by the verifier producing ALLOW. It MUST NOT treat HOLD as DENY (the authorization cycle may resume with new independent evaluation). During HOLD, the device or system remains in its safe state.
- ALLOW: the gate MAY release execution after G2 through G5 are satisfied.

A gate that overrides DENY is not PCE-compliant. A gate that releases HOLD without re-evaluation is not PCE-compliant.

Requirement G7 — Fail Closed
A compliant gate MUST fail closed on any verification error, timeout, format violation, or ambiguous state. Ambiguity is not permission. A gate that fails open — releasing execution when it cannot complete verification — is not PCE-compliant.

Requirement G8 — Independent Gate Log
A compliant gate MUST log its own release decision independently of the verifier's ProofRecord. The gate log entry must contain: the gate's own identity, the timestamp of the gate decision, the decision outcome (ALLOW released / HOLD suspended / DENY blocked), and a cryptographic hash of the received PCE package binding this log entry to the package that was evaluated. The gate log and the verifier's ProofRecord together constitute the complete evidentiary record of the authorization cycle.

---

8. The Fail-Closed Doctrine

PCE adopts a single governing stance on ambiguity: **the absence of valid proof is sufficient grounds for denial.**

This is the inverse of most authorization systems, which grant access unless explicitly denied. PCE denies unless explicitly and verifiably authorized. The default is closed.

This design choice has a cost. It means that a correctly authorized action with a malformed package will be denied. The burden of generating valid proof falls on the proposing system, not on the gate. This is intentional. The agent bears the proof obligation; the gate bears the verification obligation; the world bears no obligation to accept unproven consequence.

The fail-closed doctrine also creates a denial-of-service surface: an attacker who can corrupt, delay, or intercept PCE packages can prevent consequential actions from executing, even authorized ones. This tradeoff is deliberate. In high-consequence domains, a prevented authorized action is recoverable; an executed unauthorized action may not be. The cost of the denial surface is accepted in exchange for the guarantee that ambiguity cannot be exploited to authorize action.

PCE is a systems primitive for high-consequence domains. In low-consequence domains — queries, computations, outputs that produce no lasting change in the world — the overhead of package generation and verification may not be justified.

---

9. What Remnant Fieldworks Has Already Built

The following maps the PCE specification to the ExecutionProof architecture as of August 2026. These mappings are stated honestly: they identify what is built, what is partial, and what remains.

The ProofRecord — Most of the PCE Package
The ExecutionProof ProofRecord currently contains: action, verifier_decision, verifier_signature, and partial versions of authority_binding, evidence_set, policy_state, and constraint_envelope. Gap fields not yet implemented as standardized PCE fields: intent_lineage, freshness_nonce, and expiration. These represent the next protocol layer to be added to the existing architecture.

The Engine Stack — Most of the Verifier
The four-engine architecture (Authority → Evidence → Constraint → Control) maps directly to the PCE verifier evaluation requirements. Short-circuit logic ensures a failed Authority evaluation does not produce a misleading partial result. Gap checks not yet implemented as formal protocol components: nonce generation and registry (Requirement V6), expiration setting (Requirement V5), and the gate-independent log artifact (Requirement G8 — the verifier currently emits one record; the gate log is not yet a separate artifact).

The EPHRC — A Physical Gate Prototype
The EPHRC's `execute_enable` signal is the hardware expression of the PCE gate release decision. It does not yet consume a full PCE package. Full gate compliance would require the gate to receive, parse, and verify the package signature before asserting the signal. That integration is future work.

The FIDELITY Series — Intent Lineage Research
Six experiments (IF-01 through IF-06, Zenodo DOI 10.5281/zenodo.21927042) investigating intent fidelity across transformation steps. This is the research foundation for the future `intent_lineage` required-field specification. It is founder-led internal research; independent validation is the next phase.

Experimental Corpus
106 documented design-before-execution experiments across 12 research families: 95 PASS, 8 preserved FAIL, 3 special status, 4 validated FAIL→PASS remediations. 85 Zenodo depositions. Internal and founder-led; all claims are bounded by that provenance.

---

10. What Remains

Package format standardization. Canonical serialization format (JSON-LD or CBOR), published schema, and versioning mechanism are needed. The three gap fields (intent_lineage as future required, freshness_nonce, expiration) need implementation in the current ProofRecord.

Verifier reference implementation. A freely available, independently auditable reference verifier is required for reproducibility. The gap checks (Requirement V5, V6, and the gate-independent log) must be added.

Intent lineage verification specification. Before `intent_lineage` can become a required field, the method for verifying intent chain consistency must be specified and independently validated. The FIDELITY research series is the current foundation.

Nonce registry specification. Nonce persistence, distributed synchronization, and recovery after gate restarts require a full protocol specification.

Gate-to-gate federation. How a downstream gate handles a PCE package already verified upstream — analogous to certificate chain verification — requires protocol definition.

Provisioning annex. The verifier-recognition mechanism (Requirement G3) requires a full specification of key provisioning, storage, and rotation procedures.

Independent hardware validation. EPHRC requires physical board execution, vendor-tool signoff, and independent reproduction before the hardware gate claim can be elevated beyond prototype status.

Standards submission. Recommended primary body: IETF (internet-standards protocol model, broad reach into software systems). Submission requires this specification to first survive external technical review. The Dayton Adversarial Boundary Challenge (Fall 2026) is the first external stress test.

---

11. The Category This Creates

If PCE is adopted, the governing question for autonomous systems shifts from:

“Here is an AI command.”

to:

“Here is an AI command and the proof that gives it the right to exist.”

That is a different computing model. Under that model:

- An autonomous agent that cannot generate a valid PCE package cannot execute — not because a human intervened, but because the protocol refuses it.
- An auditor does not reconstruct what happened from logs. The ProofRecord carries the proof that was required for the action to execute.
- A regulator does not ask whether an AI system was “aligned.” It asks: where is the ProofRecord?
- A court does not debate intent. The intent_lineage field is the cryptographic record of what the human said and how it was transformed.

This is the consequence layer. PCE is the protocol proposal that names it, specifies it, and provides a partial implementation for external review.

---

12. Patent Notice

This work is related to patent applications pending before the United States Patent and Trademark Office. Applications are pending — not granted. The full portfolio contains 8 pending non-provisional utility applications and 48 provisional applications. Licensing inquiries: derek@ownerremnantfieldworks.com.

---

13. Honesty Statement

This document is a working protocol specification produced by the inventor and founder of Remnant Fieldworks Inc. It has not undergone peer review. The partial implementation described herein is internal and founder-led. The claims regarding experimental results are accurate to the extent of the current corpus; that corpus is not a substitute for independent reproduction.

The intellectual lineage claim — that PCE applies the PCC architectural pattern to the domain of consequential action — is the authors' own characterization. We believe the analogy is structurally strong and defensible; we do not claim it has been reviewed or endorsed by PCC researchers or their institutions.

We publish this specification not as a finished standard but as a contribution to the conversation that must happen before autonomous systems are trusted with consequential authority at scale.

References

Necula, G. C. (1997). Proof-Carrying Code. Proceedings of the 24th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL '97), pp. 106–119.

Necula, G. C., & Lee, P. (1996). Safe Kernel Extensions Without Run-Time Checking. Proceedings of the 2nd USENIX Symposium on Operating Systems Design and Implementation.

NIST. (2024). FIPS 204: Module-Lattice-Based Digital Signature Standard (ML-DSA). National Institute of Standards and Technology.

NIST. (2015). FIPS 202: SHA-3 Standard: Permutation-Based Hash and Extendable-Output Functions. National Institute of Standards and Technology.

Hone, D. A. (2026). Proof Before Power. [Amazon #1 Bestseller at launch.]

Remnant Fieldworks Inc. (2026). ExecutionProof Experimental Corpus — ARK, WITNESS, FIDELITY, BELLWETHER, CHRONO, TRINITY, EPHRC and related series. 85 Zenodo depositions. doi:10.5281/zenodo.22003045 (EPHRC), doi:10.5281/zenodo.21927042 (FIDELITY), and others.

Hone, D. A. (2026). EXECUTE_ENABLE: A Proposed Open Hardware Standard for Consequential Machine Interlock. Remnant Fieldworks Inc. Working Specification v0.2.

Remnant Fieldworks Inc. — Ohio C-Corp — Westerville, Ohio
remnantfieldworks.com · executionproof.io